

Final Report

Project Members: David Segal, Aviel Cohen, Alex Baranov

Project Advisor: Prof. Lee-Ad Gottlieb

August 2022

Ancient Roman Coin Classifier

Thanks

We would like to thank Professor Lee-Ad Gottlieb for the trust and independence that was given to us during this project.

To Yanniv David Levy for helping us with the data processing and categorizing and for being our source for everything coin related.

For everything, thank you.

Abstract

System that performs fast and reliable classification of ancient roman coins. Today archeologists need to observe manually and separately each ancient roman coin to classify it, we want to try to automate this process with ML tools. The classification is time consuming, takes effort and can be problematic with damaged coins. Our goal to the system is to be reliable, which means with a low number of incorrect classifications, and fast.

1 Introduction

The project consists of several parts:

1. Data Collection
 - Collecting data for the classification problem.
2. Classification:
 - Binary classifier between two types of crowns - 'laurate' and 'radiate'.
 - Multi-Class Classifier between nine possible roman emperors – 'nerva', 'hadrian', 'titus', 'trajan', 'antoninus', 'lucius', 'vespasian', 'domitian' and 'marcus'.

Given an image of a roman coin, we can classify it between either what type of crown the emperor wears or which emperor is shown on the coin.

In the project we focused mainly on part 2 (Classification), practically on the emperor classifier. Our main goal was experimenting with known and advance solutions to problems in an old fashion field that relay on old technologies, to see if we can initiate a new approach.

2 Data Collection

To be able to train a Deep Learning model the model needs to be train on a vast amount of data. The Date that was needed to the problem we were trying to solve was images of ancient roman coins form different types, eras, and conditions.

The Data collection was one of our tasks in this project. So, to be able to collect this data efficiently a python script was written – that collects images of coins from several archeologists' sites and categories them with the correct label. After that some more pre-processing, cleaning and categorizing was made by hand so there won't be any wrong labels that are passed to our model.

3 Bad Condition Coins

One of the main goals of this project was to be able classify correctly worn down and bad condition coins, coins that are hard to classify even for experts.

To be able to achieve this, the project used advanced methods in the computer vision field.

Examples form our data set



Good condition coins



Worn down coins

4 Imbalance and Overfit Problem – Emperor Classifier

Imbalance – is a common Machine Learning problem in which the data is not equally distributed, which the model assumes it is. So, when we have imbalanced classes, the Machine Learning algorithm tends to be more biased towards the majority class or classes, causing bad classification of minority classes.

Overfit – is a common Machine Learning problem in which the model has been over trained on the training set to a point where it learns features that are too specific for the training data, but not necessarily correct to new data. The model knows the training data well but does not generalize, and therefore overfitted model may fail to predict reliably. This connects to our imbalance problem, the model overfits itself for the majority class or classes.

Only with the emperor classifier we needed to use this technique, with the crowns we had enough data to use (and discard if needed) and the classification is more elementary.

4.1 Our Solution

After reading about class imbalance solution, including over and under sampling and class weights, we decided that giving our classes weights was the better solution. The class weight

works as follows: for each mistake of a sample – penalizes with its class weight instead of a default value of 1. So, for minority classes it gives a higher-class weight, and penalizes more the model.

For each class the weight was assigned accordingly:

$$\frac{\text{total number of samples}}{\text{number of classe} \times \text{number of samples for that class}}$$

So now, while training the model, it will be penalizing as if the data is balanced.

5 Different Images Size

Because the data was collected by us and was gathered from different sites, most of the images were in different resolutions. One of the main problems when training a model whose input is images, is the different size of the images. The problem results from the fact that the model needs to know what input size to expect in his input layer.

5.1 Methods for Different Image Sizes

1. Resizing all images to one certain size – this action might cause a loss of information that was in the original image.
2. Padding all images with a solid color to one certain size – this option might introduce an additional error source to the network's prediction, as the network likely will be biased to images that contain such a padded border.

5.1 Our Solution

We decide to try several image sizes and choose the one that will suit our problem best – large images will be scaled down and small images will be starched.

While training the algorithms (section 7) we also tried using different image sizes starting from 64 by 64 pixels and up to 350 by 350 pixels. We decided to settle down on 256 by 256 pixels, going higher wasn't beneficial and even hundred our model performances.

6 Making the Data fit Real Life

The images that were collected didn't fit the images that would be used in real life, i.e., most of the coin images that were collected were in perfect lighting, white background, high quality camera and there was no tilt to the coin.

If were to train the model on those images it would overfit to those examples, and if we were try to predict a label for a new coin with a bit of shade, different background, low resolution camera or rotate the coin the model most likely to misclassify the coin.

6.1 Our solution

To account for that we decided to do some augmentation to our image before training the model. The following image augmentation were preformed: Color Jittering, Blurring the image (this two for compensate shading, background, and camera quality) and rotate the image by max of -25 to 25 degrees (this compensates for the lack of tilt).

In a latter chapter (9) we discuss about an app we made to make it easier for archeologists to use the model. To make the model less vulnerable to the image crop, the images were resized to 280 by 280 and were random cropped to 256 by 256 pixels. This made the images closer to real life and made the model less vulnerable to the image crop.

7 Classifying – Finding the Right Model

To be able to correctly classify our coins, we decided to try several machine and deep learning algorithms and see which one works for our task. We saw which technique worked best on the crowns model (the simpler task between the two), and with that technique we also used it for the emperor's classification. All our models were trained with the Cross Entropy Loss Function (common cost function for classification problems).

This are the main models we trained and the results:

Model (Crowns)	Accuracy
ANN	73.2%
Multi-Layer ANN	84.6%
CNN	91.3%
Pre-trained Res-Net 18	97.7%

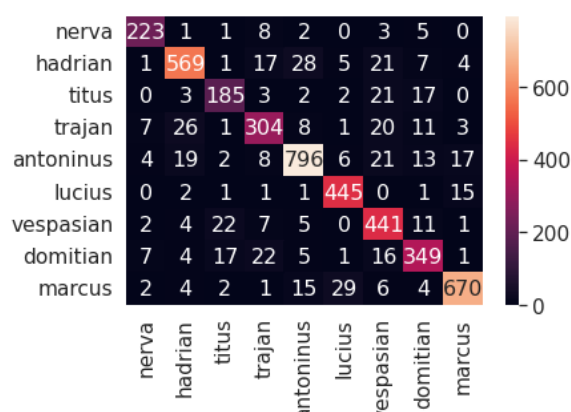
We decided that the best model to use is the Res-Net 18. After the crown classifier was trained, we made another one for the emperors using Res-Net. Experimented with different parameters and got the following results.

Pre-trained Res-Net 18	88.7%
------------------------	-------

8 Final Results

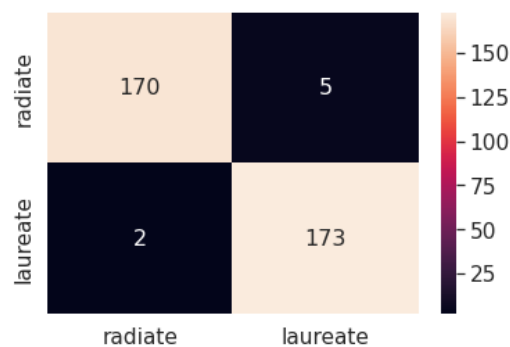
After running the models on the test set (images that the model never saw). We get the following results:

Emperors model:



We see that the model managed to learn the differences between the emperors. We also can see which emperors were misclassified with other emperors. This can help researchers in the field with analysis of those coins. And try different techniques to distinguish between them.

Crown model:



Here we also see that the model managed to learn the difference between the two types of crowns.

9 Website

After creating the model and seeing the result we wanted to do something useful with our achievements. We developed an accessible website so everyone can use just by simply entering an image that contains a roman coin and choosing which type of classification you want to make.

The website contains backend and frontend. To split the responsibility, the front interacts with the user and the backend run the models.

1. Frontend – written in JavaScript with the REACT framework and run on “netlify”.
2. Backend – written in PYTHON with the FLASK framework and run-on free version HEROKU server. Very simple and fits our needs.

The user inserts a picture and choose which type of classification he wants to make. After the user clicks on the classify button a http request is send to the server that contains picture and the type of classification.

Because we use a free version of HEROKU the server can be down after long time with no use. The request from the frontend will wake up the server, so the first request can take several seconds, maximum 20 seconds. After that, classification is made in maximum of 5 seconds. Once the backend is up, it loads the models and takes the given picture, resizes to 256 by 256 pixels, so it can be run on the model. Then runs the image on the model, makes prediction and return answer as a JSON format which contains the code 200 (meaning run successfully). The given answer will present to the user.